



Applying FinOps to Agent Costs

Welcome to the final module: Applying FinOps to Agent Costs.

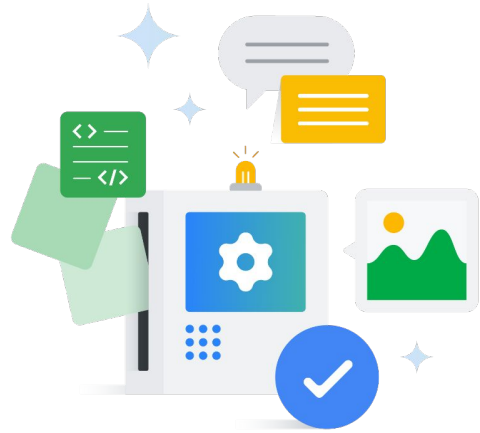
Now we turn to an operational reality of production AI systems: cost management.

AI agents can generate unpredictable costs due to token usage, model selection, tool calls, and infrastructure scaling.

This module introduces FinOps practices that help teams monitor, control, and optimize the cost of running AI agents in production environments.

In this module, you learn to ...

- 01 Identify the primary cost drivers of AI agents
- 02 Reduce token and model costs
- 03 Architect cost-efficient agent systems
- 04 Implement a measurable AI FinOps loop on Google Cloud



In this module, you'll learn four key skills.

- First, how to identify the primary cost drivers of AI agents.
- Second, how to reduce token and model costs using prompt optimization and architectural techniques.
- Third, how to design cost-efficient agent systems using routing, caching, and right-sized models.
- Finally, we'll look at how to implement a continuous FinOps loop that measures and optimizes AI costs over time.

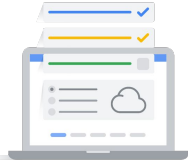
Financial governance challenges

Lack of visibility and cost allocation



AI Cost Drivers

- Unpredictable token usage
- Model tier selection
- Multi-agent orchestration
- Tools/API downstream cost
- Session-level cost invisibility



Governance Controls

- Project-level cost isolation
- Agent-level cost tagging
- Token-level monitoring
- Budget alerts per workload
- Billing export to BigQuery

One of the biggest challenges in AI systems is lack of cost visibility.

Unlike traditional applications, AI costs are often driven by token usage, which can fluctuate depending on prompts, conversation length, and agent reasoning paths.

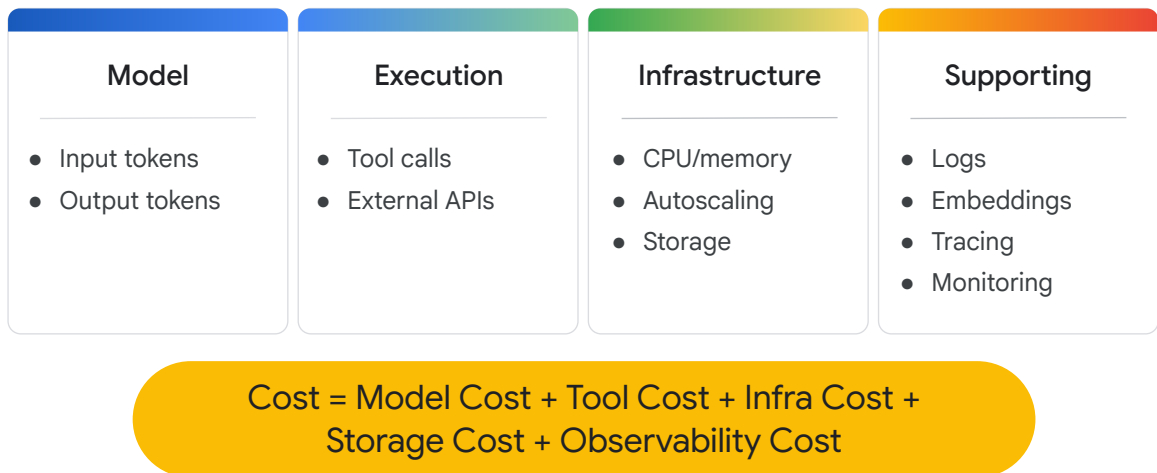
Other cost drivers include model tier selection, multi-agent orchestration, and downstream tool or API calls.

To address these challenges, organizations implement governance controls such as project-level cost isolation, agent-level tagging, token monitoring, and billing exports to analytics systems like BigQuery.

FinOps problems:

- Unpredictable token usage
- Model tier selection impacts cost exponentially
- Multi-agent orchestration increases hidden calls
- Tool calls introduce downstream cost (DB, APIs)
- Lack of session-level cost attribution

Cost anatomy of an AI agent



The total cost of running an AI agent is composed of several components.

1. model cost, which is typically based on input and output tokens.
2. tool cost, which includes calls to external APIs or services.
3. infrastructure cost, including compute resources, scaling, memory usage, and storage cost, such as embeddings or vector databases.
4. Finally, observability systems—including logging, tracing, and monitoring—also contribute to the total cost.

Understanding this cost anatomy helps teams identify optimization opportunities.

Prompt optimization (reduce token cost)

- Prompt engineering discipline
 - Reduce token counts
 - Shorter system prompts
 - Remove redundant instructions
 - Avoid unnecessary examples
 - Structured output instead of verbose text
- Input preprocessing
 - Trim conversation history
 - Chunk retrieval results
 - Filter irrelevant context
 - Limit RAG top-K

Token count = linear cost multiplier
Model tier is often a significant cost multiplier

Model tier is often a significant cost multiplier (higher-tier models can be several times more expensive).

One of the simplest and most effective ways to reduce cost is prompt optimization.

Token count directly impacts model cost, so reducing unnecessary tokens can significantly lower expenses.

This includes shortening system prompts, removing redundant instructions, and avoiding unnecessary examples.

Other techniques include trimming conversation history, limiting RAG retrieval results, and filtering irrelevant context.

It's important to remember that token count acts as a linear cost multiplier, while model tier selection acts as an exponential multiplier on cost.

Limit RAG top-K defines the maximum number of most relevant documents, chunks, or passages the retriever fetches from a vector database to answer a user prompt.

Response caching

- Response caching can lower latency and costs by reducing LLM calls
- Google ADK provides a built-in mechanism called **Context Caching**
 - Caches the LLM prompt and response context
 - Especially useful for repeated instructions or heavy multi-turn sessions
 - Enabled by wrapping the root agent in an App and passing a ContextCacheConfig object
 - The caching is configured for the entire application
 - Cache is application-level, not sub-agent level
 - Will cache the context of the entire LLM call
 - If interactions with sub-agent2, sub-agent3, etc. lead to repetitive overall prompt contexts, you will see performance and cost benefits

Best ROI when repeated prompts share
identical system + context

Another powerful optimization technique is response caching.

ADK provides a built-in context caching mechanism (availability depends on framework and deployment setup).

If the same prompt is sent repeatedly, the system can return a cached response instead of calling the model again.

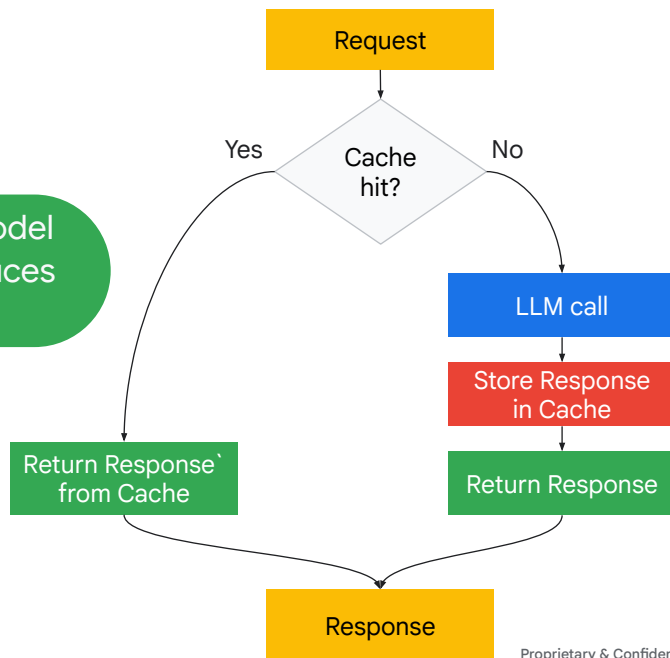
This results in both lower latency and zero model cost for those requests.

The ADK framework provides built-in support for context caching, which stores prompt and response contexts for reuse.

Caching is particularly useful when multiple sessions share identical system prompts or instructions.

Response caching

Cache hit eliminates model inference cost and reduces latency



Cache hit eliminates model inference cost and reduces latency, but may still incur minimal storage and infrastructure costs.

This diagram illustrates how caching works.

- When a request is received, the system checks whether the response is already stored in the cache.
- If a cache hit occurs, the response is returned immediately without invoking the model.
- If there is no cache hit, the system calls the model and stores the result for future reuse.
- This simple mechanism can dramatically reduce both cost and latency.

ADK context caching example

```
from google.adk.agents.context_cache_config import ContextCacheConfig
from google.adk.apps.app import App

# Your existing agent setup
myAgent = LlmAgent(...)

# 1. Define the cache configuration
cache_config = ContextCacheConfig(
    min_tokens=4096, # Only cache if the context is larger than this
    ttl_seconds=1800, # Cache items expire after 30 minutes
    cache_intervals=4 # Force a refresh every 4 invocations
)

# 2. Wrap your root agent in an App with the cache config
supervisor_app = App(
    name="supervisor_app",
    root_agent=myAgent,
    context_cache_config=cache_config,
)
```

Here we see a Python example of enabling context caching in ADK.

The configuration defines parameters such as the minimum token threshold, cache expiration time, and refresh interval.

Once configured, the agent application automatically manages cached contexts.

This allows developers to implement caching without significantly modifying application logic.

Model right-sizing (adequacy)

- Use smallest model that meets quality threshold
- Use eval scores to justify model choice
- Avoid defaulting to the largest model
- Consider latency + cost trade-offs
- Validate with evaluation metrics

Most production tasks do not require frontier models
(cheapest, fastest, good enough model)

Another important cost strategy is model right-sizing.

Not every task requires the most advanced or expensive model.

In many cases, smaller models can perform the task just as effectively.

Teams should use evaluation metrics to determine the smallest model that meets the required quality threshold.

This approach reduces both cost and latency while maintaining acceptable performance.

Fine tune smaller models

- Replace expensive base models for repetitive tasks
- Improves determinism
- Can reduce prompt complexity and improve efficiency
 - Token usage impact depends on implementation
- Lower per-request cost
- Evaluate ROI (training cost vs usage volume)
 - Fine-tuning increases training cost upfront
 - Best ROI when request volume is high
- Best for high-frequency workflows

$$\text{Break-even volume} = \frac{\text{Training Cost}}{\text{Savings per request}}$$

Fine-tuning smaller models is another strategy for reducing cost.

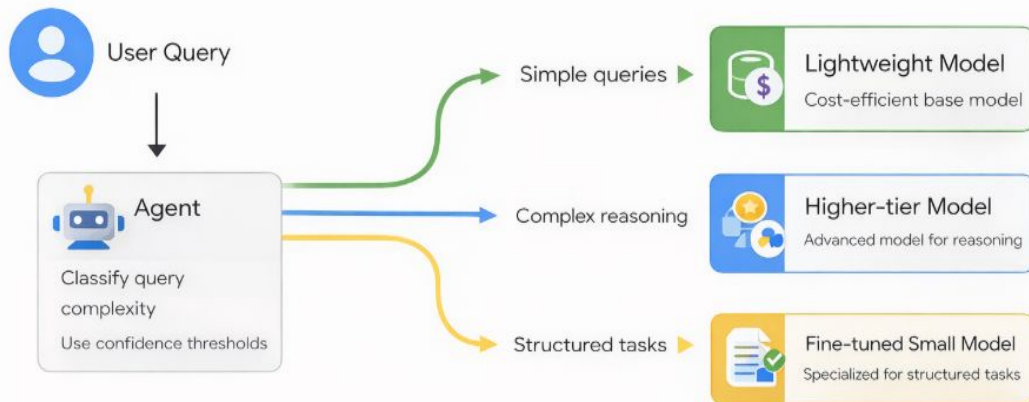
Instead of using a large general-purpose model, organizations can fine-tune a smaller model for specific tasks.

This improves determinism and reduces token usage.

However, fine-tuning introduces upfront training costs, so it is most beneficial when the system processes high volumes of requests.

The goal is to reach a break-even point where savings per request exceed the training investment.

Dynamic model routing



Dynamic model routing is a powerful architecture pattern for cost optimization.

In this approach, the system selects different models depending on the complexity of the task.

Simple queries may be routed to lightweight models.

Complex reasoning tasks may require more powerful models.

Structured or repetitive tasks might use specialized fine-tuned models.

This allows organizations to balance performance, latency, and cost.

Dynamic model routing: Three ways to achieve

01

Router Agent (Dispatcher Pattern)

Lightweight “router” agent analyzes the intent and delegates work to a specialized “sub-agent” running a different model.
Best for multi-agent orchestration.

02

Model Overrides in Code

Override the default model on a per-request basis.
Best for simple policy-based routing.



Tool-Based Model Switching

Define a tool that is itself an LLM call to a different model.
Best when framework is already tool-centric.

There are several ways to implement dynamic routing.

One option is a router agent, which analyzes the request and delegates work to specialized sub-agents.

Another approach is model overrides in code, where the model is changed programmatically for specific requests.

A third approach uses tool-based routing, where different models are exposed as tools within the agent framework.

Each method provides a flexible way to control model usage and optimize costs.

Dynamic model overrides in code example

```
from google.adk import Agent

my_agent = Agent.load("my_agent_dir")

# Query 1: Use the default model (e.g., Gemini Flash Lite)
response1 = my_agent.run("What is the sky blue?")

# Query 2: Force a more powerful model for a complex task
response2 = my_agent.run(
    "Analyze this 50-page legal document for risks.",
    config={"model_name": "gemini-2.5-pro"}
)
```

This slide shows a code example demonstrating dynamic model selection.

For simple queries, the agent uses the default lightweight model.

For more complex tasks—such as analyzing large documents—the system explicitly switches to a more capable model.

This approach allows developers to control model usage directly within the application logic.

Batch inference

- Batch inference is a throughput optimization
 - Combine multiple prompts into a single request
 - Use for evaluation, backfills, document processing
 - Improves throughput and operational efficiency
 - Cost benefits depend on pricing model and workload
 - More efficient throughput
- Not suitable for real-time (latency-sensitive) interactive agents
- Use for:
 - Offline processing
 - Evaluation jobs
 - Embedding generation
 - Large-scale document processing

Batch inference is the process of sending multiple inputs to a model at the same time and having the model process them together in a single request.

Instead of:

Prompt 1 → model → response

Prompt 2 → model → response

Prompt 3 → model → response

You send:

[Prompt 1, Prompt 2, Prompt 3] → model → [Response 1, Response 2, Response 3]

This allows the system to process many tasks more efficiently.

It is generally not suitable for real-time interactive agents due to increased latency.

Standard FinOps practices on Google Cloud

- Operational FinOps on Google Cloud
 - Budget alerts per project
 - Billing export to BigQuery
 - Cost allocation via labels
 - Separate dev/staging/prod
 - Separate AI workloads into dedicated billing projects
 - Label by model, agent, and feature
 - Monitor cost per conversation/session
 - Enforce model allowlists in production

Google Cloud supports several FinOps practices that help organizations manage AI costs.

These include budget alerts, billing exports to BigQuery, and cost allocation using labels.

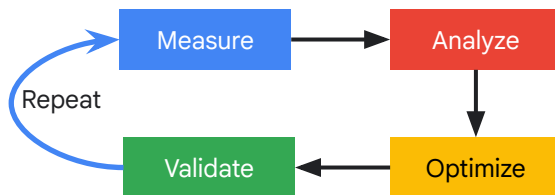
It's also recommended to separate development, staging, and production workloads into different billing projects.

Additionally, labeling costs by model, agent, or feature allows teams to monitor cost per conversation or session.

Organizations can also enforce model allowlists to prevent teams from accidentally using expensive models in production.

AI FinOps loop

- Measure
 - Token usage
 - Model mix
 - Cost per session
- Analyze
 - Billing export (BigQuery)
 - Session cost patterns
- Optimize
 - Routing
 - Caching
 - Prompt reduction
- Validate
 - Eval metrics
 - Quality score thresholds
 - Regression detection
 - Model performance drift



Google Cloud

Proprietary & Confidential 16

The point here is to make sure we realize - FinOps is not a one-time activity—it is a continuous optimization cycle.

- The loop begins with measurement, including token usage, model mix, and cost per session.
- Next is analysis, using tools like BigQuery to identify patterns in usage and cost.
- Then teams optimize the system through techniques like caching, routing, and prompt reduction.
- Finally, they validate the results using evaluation metrics and quality thresholds.

This cycle repeats continuously to ensure both cost efficiency and performance.

Questions and answers





Thank you.
Module 6 is complete.